

ALUMNAYAN · TECHNICAL DOCUMENTATION

Cohort Outcomes Feature

Per-Batch-Year Breakdown of Employment & Course-Alignment Rates

System: AlumNayan Alumni Database Module: Dashboard · Reports · PDF Export

Date: March 2026 Version: 1.0

Table of Contents

- 1. Purpose & Background pg. 1
- 2. Key Concepts & Definitions pg. 1
- 3. Computation Logic pg. 2
- 4. Column Definitions pg. 3
- 5. Data Flow & Architecture pg. 3
- 6. Files Modified pg. 4
- 7. Core Utility: `computeCohortBreakdown()` pg. 4
- 8. Dashboard Integration pg. 5
- 9. Reports Page Integration pg. 6
- 10. PDF Export — Section C pg. 6
- 11. Edge Cases & Limitations pg. 7
- 12. Sample Output pg. 7

1. Purpose & Background

The professor needs to track alumni employment and course-alignment outcomes **grouped by graduation cohort** (batch year), not just by broad career stage buckets. The goal is to answer questions like:

- "Batch 2018 — what is their employment status 5 years after graduation (2023)?"
- "Batch 2019 — 5-year check-in is due in 2024. What is the current data?"
- "Which cohort has the highest course-alignment rate?"

IMPORTANT DESIGN DECISION

The system stores only the

CURRENT

employment status of each alumni — there are no historical snapshots. Therefore, the table shows each cohort's *current recorded status* while framing it with a "5-Year Evaluation Year" column (batch year + 5) that tells the professor when a formal survey was due for that cohort.

The feature was added to three areas of the system: the **Admin Dashboard**, the **Reports Page** (UI table), and the **PDF Export** (Section C).

2. Key Concepts & Definitions

Batch Year

The year an alumni graduated (or the cohort year they are associated with). Stored as an integer in Firestore under the `batchYear` field on each user document.

5-Year Evaluation Year (Eval Year)

The year a cohort is due for a formal 5-year outcome survey, computed as:

$$\text{Eval Year} = \text{Batch Year} + 5$$

Example: Batch 2018 → Eval Year 2023. This matches the institution's survey cycle where graduates are formally tracked 5 years after their graduation.

Years Out

How many years have passed since the cohort's graduation as of the current year:

$$\text{Years Out} = \text{Current Year} - \text{Batch Year}$$

This gives context about how mature the cohort is (e.g., a batch year of 2020 is currently 6 years out in 2026).

Course-Aligned Employment

An alumni is considered **course-aligned** if they are both employed *and* their job is related to their engineering program. Alignment is determined by:

1. **Direct survey answer** — if the alumni explicitly answered a `courseAligned` yes/no field in their profile, that answer takes priority.
2. **Keyword matching (fallback)** — if no direct answer exists, the system checks the alumni's `currentPosition` job title against a predefined keyword list for their program (e.g., "logistics", "lean" for Industrial Engineering).

WHY SURVEY ANSWER TAKES PRIORITY

Keyword matching can produce false positives (e.g., a job title that mentions "industrial" but is unrelated to IE). When alumni explicitly confirm alignment, that signal is more accurate. Keyword matching serves as a fallback for older records.

3. Computation Logic

Step 1 — Group alumni by batch year

The function iterates over all alumni documents. Alumni with no `batchYear` are skipped. For each valid alumni, a running tally is maintained per batch year:

- **total** — count of all alumni in that cohort
- **employed** — count where `isEmployed === true`
- **aligned** — count where the alumni is both employed and course-aligned (via the `isAligned()` helper)

Step 2 — Compute rates

Both rates are rounded to the nearest whole percent:

$$\text{Employment Rate} = \text{round}(\text{employed} / \text{total} \times 100)$$

$$\text{Alignment Rate} = \text{round}(\text{aligned} / \text{total} \times 100)$$

DENOMINATOR NOTE

The

ALIGNMENT RATE

denominator is the full cohort (`total`), not just employed alumni. This means it represents the percentage of all graduates in that cohort who are both employed and in a course-aligned job — a more conservative and meaningful metric for accreditation purposes.

Step 3 — Sort and return

Rows are sorted **newest batch year first** (descending), so the most recent cohorts appear at the top of the table.

4. Column Definitions

Column	Source / Formula	Description
Batch Year	alumni.batchYear	The graduation cohort year
5-yr Eval Year	batchYear + 5	Year the 5-year survey is due for this cohort
Years Out	currentYear – batchYear	How many years since graduation (as of today)
Total	count(alumni where batchYear = N)	Number of alumni in the cohort
Employed	count(isEmployed === true)	Alumni confirmed as currently employed
Employment Rate	round(employed / total × 100)	% of cohort that is employed
Course-Aligned	count(isAligned(a) === true)	Employed alumni whose job matches their program
Alignment Rate	round(aligned / total × 100)	% of cohort that is employed and course-aligned

5. Data Flow & Architecture

```
// High-level data flow [ Firestore "users" collection ] |
getDocs(query(where("role","==","alumni")) ) ▼ [ alumniDocs: UserDoc[] ] |
computeCohortBreakdown(alumniDocs) ▼ [ Group by batchYear → Map<number, {total,
employed, aligned}> ] | isAligned(a) – survey answer or keyword match ▼ [ CohortRow[]
– sorted newest first ] | ↳ Dashboard: setCohortRows() → <table> render | ↳
Reports UI: useMemo(cohortRows) → <table> render | ↳ PDF Export: autoTable(doc, {
body: cohortRows.map(...) })
```

isAligned() Helper

This private function (in `courseAlignment.ts`) implements the alignment check:

```
function isAligned(a: AlumniForOutcome): boolean {
  if (!a.isEmployed) return false;           // not employed → never aligned
  if (typeof a.courseAligned === "boolean") // explicit survey answer?
    return a.courseAligned;
  // fallback: keyword match on job title
  return isCourseAligned(a.currentPosition ?? "", a.course);
}
```

6. Files Modified

File	Change
src/lib/utils/courseAlignment.ts	Added <code>CohortRow</code> interface and <code>computeCohortBreakdown()</code> function
src/app/(admin)/admin/dashboard/page.tsx	Import new utilities; added <code>cohortRows</code> state; call <code>computeCohortBreakdown</code> in <code>load()</code> ; added Cohort Outcomes table card
src/app/(admin)/admin/reports/page.tsx	Import new utilities; <code>cohortRows</code> via <code>useMemo</code> ; added Cohort Outcomes UI card; added Section C to PDF export

7. Core Utility: `computeCohortBreakdown()`

Located in `src/lib/utils/courseAlignment.ts` . Full implementation:

```
export interface CohortRow {
  batchYear:    number; // graduation cohort year
  evalYear:     number; // batchYear + 5
  yearsOut:     number; // currentYear - batchYear
  total:        number; // all alumni in cohort
  employed:     number; // isEmployed === true
  employmentRate: number; // % employed (0-100)
  aligned:      number; // employed + course-aligned
  alignmentRate: number; // % aligned of total (0-100)
}

export function computeCohortBreakdown(
  alumni: AlumniForOutcome[]
): CohortRow[] {
  const currentYear = new Date().getFullYear();
  const map = new Map<number, { total: number; employed: number; aligned: number }>();

  for (const a of alumni) {
    if (!a.batchYear) continue; // skip alumni with no batch year
    if (!map.has(a.batchYear))
      map.set(a.batchYear, { total: 0, employed: 0, aligned: 0 });
    const row = map.get(a.batchYear)!;
    row.total++;
    if (a.isEmployed === true) row.employed++;
    if (isAligned(a)) row.aligned++;
  }

  return Array.from(map.entries())
    .sort(([a], [b]) => b - a) // newest batch year first
    .map(([batchYear, row]) => ({
```

```
    batchYear,
    evalYear:      batchYear + 5,
    yearsOut:      currentYear - batchYear,
    total:         row.total,
    employed:      row.employed,
    employmentRate: row.total > 0
      ? Math.round((row.employed / row.total) * 100)
      : 0,
    aligned:       row.aligned,
    alignmentRate: row.total > 0
      ? Math.round((row.aligned / row.total) * 100)
      : 0,
  }));
}
```

Input type: `AlumniForOutcome`

Field	Type	Usage
batchYear	number null	Groups the alumni into a cohort row. Null → skipped.
course	string null	Used by keyword-matching fallback for alignment.
isEmployed	boolean undefined	Counted toward <code>employed</code> .
currentPosition	string undefined	Job title for keyword-matching alignment fallback.
courseAligned	boolean undefined	Explicit survey answer for alignment (highest priority).

8. Dashboard Integration

File: `src/app/(admin)/admin/dashboard/page.tsx`

STATE

```
const [cohortRows, setCohortRows] = useState<CohortRow[]>([]);
```

DATA LOADING (INSIDE USEEFFECT → LOAD())

```
// After fetching all alumni docs:
setOutcomeRates(computeOutcomeRates(alumniDocs));
setCohortRows(computeCohortBreakdown(alumniDocs)); // <-- new
```

RENDER LOCATION

The Cohort Outcomes card is placed **between the Charts section and the Recent Sign-ups card**. It is only rendered when `cohortRows.length > 0` (i.e., there is at least one alumni with a batch year).

LAYOUT ORDER ON DASHBOARD

KPI cards → Employment Overview / Course-Aligned Outcome Rates → Alumni by Course chart / Employment by Course chart →

COHORT OUTCOMES BY BATCH YEAR

(new) → Recent Sign-ups

9. Reports Page Integration

File: `src/app/(admin)/admin/reports/page.tsx`

MEMOIZED COMPUTATION

```
const cohortRows = useMemo(
  () => computeCohortBreakdown(results),
  [results]
);
```

`results` is the array of `UserDoc` objects returned from Firestore after the user clicks **Generate Report**. The memo recalculates whenever the results change.

FILTER BEHAVIOR

Filter applied	Cohort table shows
All years, all programs	One row per batch year that has data
Specific batch year (e.g., 2018)	Single row for Batch 2018 only
Specific program, all years	Per-batch-year rows for that program only
Program + batch year	Single row for that program + year combination

RENDER LOCATION

The Cohort Outcomes card appears **between the 7 stat-cards and the Preview (alumni registry) card**, inside the `generated && results.length > 0` conditional block.

10. PDF Export — Section C

The PDF export in `exportPDF()` is structured in three sections on Page 1, followed by the Alumni Registry on Page 2+.

Section	Content
A	Employment Summary — Employed / Unemployed / Total with counts and rates
B	Course-Aligned Placement Rates — by career stage bucket (0–2 yrs, 3–5 yrs, 6+ yrs)
C (new)	Cohort Outcomes by Batch Year — the per-batch breakdown table

PLACEMENT IN THE PDF

```
// Section C starts immediately after Section B's last row
const afterB = (doc as any).lastAutoTable.finalY + 8;
doc.text("C. Cohort Outcomes by Batch Year", 14, afterB);
autoTable(doc, {
  startY: afterB + 3,
  head: [
    ["Batch Year", "5-yr Eval Year", "Years Out",
      "Total", "Employed", "Employment Rate",
      "Course-Aligned", "Alignment Rate"]],
  body: cohortRows.map((row) => [
    String(row.batchYear), String(row.evalYear),
    `${row.yearsOut} yrs`, String(row.total),
    String(row.employed), `${row.employmentRate}%`,
    String(row.aligned), `${row.alignmentRate}%`,
  ]),
  styles: { fontSize: 8 },
  headStyles: { fillColor: [30, 41, 82] }, // navy
  columnStyles: {
    5: { fontStyle: "bold" }, // Employment Rate — bold
    7: { fontStyle: "bold" }, // Alignment Rate — bold
  },
});
```

PAGE OVERFLOW BEHAVIOUR

jspdf-autotable automatically adds new pages if Section C overflows the first page. The footer note is fixed at y = 285 on page 1, so it may overlap the table if there are many batch year rows. This is acceptable for typical cohort sizes (< 20 rows).

11. Edge Cases & Limitations

Scenario	Behaviour
Alumni with no <code>batchYear</code>	Skipped entirely — not included in any cohort row
Alumni with <code>isEmployed</code> not set (undefined)	Not counted as employed; not counted as aligned

Scenario	Behaviour
Cohort with 0 alumni (after filtering)	No row produced (division by zero is guarded: rate returns 0)
Current year cohort (graduated this year)	Years Out = 0, Eval Year = current year + 5; included normally
Future batch years in the database	Years Out would be negative — included as-is; no filtering is applied
No alumni have batch years at all	Dashboard: table card is not rendered (<code>cohortRows.length === 0</code>)
Keyword matching ambiguity	Mitigated by prioritising the explicit <code>courseAligned</code> survey field

NO HISTORICAL SNAPSHOTS

Because the system only stores current status, the "5-Year Evaluation Year" column is informational only. The data shown for Batch 2018 reflects their status *today*, not what it was in 2023. Future versions may introduce snapshot storage to capture point-in-time data.

12. Sample Output

Example cohort table for an institution with alumni from Batch 2018–2022 (as of 2026, using hypothetical data):

Batch Year	5-yr Eval Year	Years Out	Total	Employed	Employment Rate	Course-Aligned	Alignment Rate
2022	2027	4 yrs	18	12	67%	8	44%
2021	2026	5 yrs	24	20	83%	15	63%
2020	2025	6 yrs	31	27	87%	19	61%
2019	2024	7 yrs	28	25	89%	17	61%
2018	2023	8 yrs	22	20	91%	14	64%

Reading the table: "Batch 2021's 5-year check-in was due in 2026. As of today, 83% are employed and 63% of the entire cohort is employed in a course-aligned position."

AlumNayan · College of Engineering Alumni Database System · Documentation generated March 2026